



FACULTY OF ENGINEERING AND TECHNOLOGY
DEPARTMENT OF COMPUTER ENGINEERING

Name	Matricule
FUNWI KELSEA NDOHNWI	FE23A066
NGADANG ASSONFACK P. ESTELLE	FE23A109
ATEH SWIRRI FOFANG	FE23A018
TABOT GHISLAIN OROCK	FE23A146

**Project: AI-based Car Fault Diagnosis Mobile Application
(CheckAR)**

GROUP 11

CEF440: INTERNET PROGRAMMING AND MOBILE PROGRAMMING

INSTRUCTOR: Dr. VALERY NKEMENI

Date: May 2026

Requirements Analysis	1
General Introduction.....	2
1.0 Review and Analysis of Functional Requirements	2
1.1 Introduction.....	2
1.2 Review of Raw User Responses	2
1.2.1 Raw Functional Requirements from User Responses	2
1.2.2 Raw Non-Functional Requirements from User Responses.....	3
1.3 Analysis of Requirements	3
1.3.1 Finalized Functional Requirements with Explanations	3
1.3.2 Finalized Non-Functional Requirements with Explanations	4
2.0 Identify Issues in Requirements	4
2.1 Introduction to Issues Identification	5
2.2 Summary of Identified Issues	5
2.3 Conclusion to Issues Identification	5
3.0 Prioritization of Requirements Based on Importance and Feasibility	5
3.2 Summary of Prioritization	6
4.0 Requirements Classification and Specifications	6
4.1 Purpose of Classifying Requirements	7
4.2 Functional Requirements Specifications	7
4.3 Non-Functional Requirements	8
Performance	9
Reliability.....	9
Usability	9
Security.....	9
Maintainability & Portability	9
Quality Attributes	9
5. Software Requirements Specification (SRS)	10
5.1 Introduction to SRS	11
5.1.1 Purpose	11
5.1.2 Scope.....	11
5.1.3 Definitions, Acronyms, and Abbreviations	11
5.1.4 References	11
5.1.5 Overview	12
5.2 Overall Description	12
5.2.1 Product Perspective	13
5.2.2 Product Functions	13
5.2.3 User Classes and Characteristics	13
5.2.4 Operating Environment	14
5.2.5 Design and Implementation Constraints	14

5.2.6 User Documentation.....	15
5.2.7 Assumptions and Dependencies	15
5.3 Specific Requirements	15
5.3.1 Functional Requirements	16
5.3.2 External Interface Requirements	17
5.3.3 Non-Functional Requirements	19
5.4 System Features	20
5.4.1 User Authentication System.....	21
5.4.2 Dashboard Warning Light Scanner	21
5.4.3 Engine Sound Analyzer.....	21
5.4.4 Diagnosis and Reporting	22
5.4.5 Maintenance Tracking	22
5.4.6 Online / Offline Functionality	23
5.5 Validation and Testing	23
5.5.1 Testing Strategy	24
5.6 Appendices.....	24
A. Glossary	25
B. Analysis Models	25
C. Issue Tracking.....	25
6. Requirements Validation with Stakeholders.....	26
6.1 Validation and Risk Mitigation.....	27
6.1.1 Stakeholder Engagement.....	27
6.1.2 Identified Risks and Mitigation Strategies	27
General Conclusion	27

Requirements Analysis

General Introduction

Requirements analysis is the critical bridge between stakeholder aspirations and a successful software solution. In the context of our AI-powered car fault diagnosis app CheckAR, this phase has gone far beyond capturing a wish list — it has rigorously validated and refined every feature through stakeholder workshops, surveys of automotive mechanics, expert interviews, brainstorming sessions, and analysis of existing diagnostic tools. By dissecting user stories, mapping them to technical capabilities, and stress-testing each requirement for feasibility, clarity, and consistency, we have ensured that every line in our specification is both actionable and aligned with real-world needs. This disciplined approach minimizes downstream surprises, tightly focuses development effort on high-impact functionality, and lays a rock-solid foundation for design and implementation.

1.0 Review and Analysis of Functional Requirements

1.1 Introduction

This report outlines the requirements for CheckAR, a car diagnostic mobile application, based on user responses gathered through a structured survey administered to automotive mechanics and car owners. The survey targeted common car problems, misunderstood dashboard warning lights, diagnosis approaches, and expectations for a mobile diagnostic tool. The report categorizes requirements into functional and non-functional aspects, following a two-part structure:

- Review of raw user responses
- Analysis (based on completeness, clarity, technical feasibility, and dependency relationships) of requirements

1.2 Review of Raw User Responses

1.2.1 Raw Functional Requirements from User Responses

- Scan and interpret dashboard warning lights using the device camera
- Record and analyze engine sounds to detect mechanical faults
- Provide automated explanations and repair suggestions based on diagnosis
- Provide access to video tutorials for common car repairs
- Display instant diagnosis results immediately after analysis
- Offer detailed repair recommendations based on the specific fault detected
- Send maintenance reminders based on vehicle condition or schedule
- Log maintenance records and associated costs
- Support live chat with a mechanic or support representative
- Provide step-by-step guides for vehicle troubleshooting and repair
- Show maintenance alerts when critical thresholds are detected
- Diagnose all common and critical faults in a car
- Perform predictive maintenance using machine learning to anticipate potential issues
- Enable community forums and peer support interaction among users

- Provide early detection and prediction of faults before they escalate

1.2.2 Raw Non-Functional Requirements from User Responses

- Accuracy of the diagnosis (near-perfect expected by most respondents)
- Simplicity and low complexity of the technology interface
- Low data usage and minimal cost
- Zero advertisement policy
- Efficiency in response time and battery use

1.3 Analysis of Requirements

The requirements were assessed against four criteria:

- Completeness: whether enough information is provided — do the requirements cover all aspects of what the system needs to do?
- Clarity: whether the requirement is easy to understand — is each requirement clearly expressed without ambiguity?
- Technical feasibility: whether it is realistically implementable — can these requirements be delivered with available mobile technology?
- Dependency relationships: whether one requirement depends on others — how do requirements relate to and impact each other?

1.3.1 Finalized Functional Requirements with Explanations

1) Scan and interpret dashboard warning lights

Use image recognition (computer vision / CNN model) to detect and explain dashboard indicators, including urgency classification.

2) Record and analyze engine sounds

Capture engine noises via microphone and identify fault patterns such as knocking, squealing, or hissing using on-device ML algorithms (MFCC + CNN/LSTM).

3) Provide automated explanations and repair suggestions

Generate readable diagnostic reports with likely causes and practical solutions, consolidated from overlapping raw requirements.

4) Provide access to video tutorials for repairs

Offer curated repair videos for common faults, linked from diagnostic results via YouTube API integration.

5) Display instant diagnosis results after analysis

Show a quick, digestible summary of diagnostics as soon as analysis completes.

6) Send maintenance reminders based on vehicle condition or schedule

Notify users of upcoming maintenance tasks based on mileage intervals, elapsed time, or previously detected fault urgency.

7) Log maintenance records and maintenance cost

Maintain a timestamped history of repairs, diagnostics, and estimated cost for future reference and analytics.

8) Support live chat with a mechanic or support representative

Provide real-time communication with automotive professionals or an AI-assisted chat for consultation.

9) Show maintenance alerts when critical thresholds are detected

Trigger immediate warnings when engine or system parameters exceed safe operating limits.

10) Perform predictive maintenance using machine learning

Use historical diagnostic data trends to forecast problems before they cause failures.

11) Enable community forums and user interaction

Allow users to post questions, share experiences, and interact about vehicle issues and repairs.

12) Provide early detection and prediction of faults before they escalate

Continuously monitor collected data and alert users at the first signs of a developing problem.

1.3.2 Finalized Non-Functional Requirements with Explanations

1) Accuracy of the diagnosis

CheckAR shall provide diagnostic results with at least 90% accuracy for visual indicators and at least 85% for audio classification under typical conditions.

2) Simplicity and low interface complexity

The UI/UX design must enable non-technical users to complete a full diagnosis within three taps from the home screen, with no automotive jargon in plain-language outputs.

3) Data usage and cost optimization

The app shall minimize bandwidth consumption through lightweight data formats, compressed model updates, and local caching of diagnostic results.

4) Zero advertisement policy

No in-app advertisements shall be displayed at any point to maintain user trust, focus, and professional presentation.

5) Efficiency

All diagnostic functions shall complete within acceptable timeframes ($\leq 5s$ image, $\leq 7s$ audio), with minimal battery and CPU consumption during operation and standby.

2.0 Identify Issues in Requirements

2.1 Introduction to Issues Identification

High-quality, unambiguous requirements are the foundation of any successful software project. To ensure CheckAR meets both user and technical needs, we conducted structured peer reviews and walkthroughs with the development team, systematically identifying and clarifying problematic statements before design and implementation begin.

2.2 Summary of Identified Issues

Inconsistencies:

We discovered overlapping repair-guidance requirements. Specifically, 'Offer detailed repair recommendations based on diagnosis' and 'Provide step-by-step guides for vehicle troubleshooting and repair' both duplicate content covered by 'Provide automated explanations and repair suggestions,' leading to redundant functionality that has been consolidated into a single finalized requirement (FR9 + FR19).

Ambiguities:

The raw requirement 'Diagnose all common and critical faults in a car' is overly broad. The scope of 'all faults' and the distinction between 'common' and 'critical' varies significantly by vehicle make, model, year, and geographic market. CheckAR's scope has been refined to focus on fault categories that are diagnosable through visual (dashboard camera) and audio (engine sound) inputs, with explicit disclaimer that OBD-II-level hardware integration is out of scope.

Missing Information:

Our review confirmed no critical gaps remain in the documented requirements. All stakeholder needs expressed in the survey appear captured in the finalized list. The primary focus during review was on refining existing statements for clarity and consistency rather than adding new requirements.

2.3 Conclusion to Issues Identification

By identifying and resolving these inconsistencies and ambiguities early — through collaborative peer reviews and targeted queries to survey respondents — we have strengthened the specification, reduced the risk of downstream rework, and paved the way for an efficient and user-aligned development phase.

3.0 Prioritization of Requirements Based on Importance and Feasibility

To guide an efficient development roadmap for CheckAR, requirements were prioritized using the MoSCoW method:

- **Must-Have:** Critical for core functionality and user value.

- Should-Have: Important but not critical; adds significant value.
- Could-Have: Useful enhancements; implementable in later releases.
- Won't-Have (for now): Out of scope or not feasible in the current development phase.

Requirement	Priority	Justification
Scan and interpret dashboard warning lights	Must-Have	Foundational functionality; feasible via image recognition (CNN)
Record and analyze engine sounds	Must-Have	Core ML-based feature for fault detection
Provide automated explanations and repair suggestions	Must-Have	Essential for user understanding and self-action
Display instant diagnosis results after analysis	Must-Have	Enhances user experience and immediacy
Show maintenance alerts when thresholds are detected	Must-Have	Preventive maintenance with direct safety impact
Provide early detection and prediction of faults	Must-Have	Aligns with preventive diagnostics goal
Perform predictive maintenance using machine learning	Should-Have	High value but requires historical data and advanced modelling
Send maintenance reminders based on condition/schedule	Should-Have	Enhances utility; implementable with scheduled triggers
Log maintenance records and maintenance cost	Should-Have	Improves record-keeping and analytics capabilities
Provide access to video tutorials for repairs	Could-Have	Valuable for user guidance; depends on content integration
Support live chat with mechanic/support representative	Could-Have	Useful but operationally costly for initial version
Enable community forums and user interaction	Could-Have	Promotes engagement but not essential at MVP stage

3.2 Summary of Prioritization

This analysis confirms the technical and user-aligned prioritization of functional and non-functional requirements for CheckAR. Stakeholder feedback gathered from the mechanic survey has been effectively integrated to ensure that development efforts focus first on high-impact, technically feasible functionalities, with enhancement features planned for subsequent releases post-MVP.

4.0 Requirements Classification and Specifications

4.1 Purpose of Classifying Requirements

Classifying requirements into functional and non-functional categories ensures a clear understanding of what the system must do (functional) versus how well it must perform (non-functional). This classification:

- Guides the design and implementation process
- Ensures all user needs and technical constraints are addressed
- Facilitates testing, validation, and system maintenance
- Aids in resource allocation and prioritization

4.2 Functional Requirements Specifications

ID	Functional Requirement	Specification
FR1	User Registration	User registration with email verification step to confirm identity.
FR2	Role-based Authentication	Role-based login access (Car Owner, Mechanic, Administrator).
FR3	Password Recovery	Secure password recovery via email or security questions.
FR4	Session Management	User sessions with inactivity timeout to ensure security.
FR5	Image Capture	Users can photograph dashboard; app detects warning icons automatically.
FR6	Icon Interpretation	App explains detected icons with urgency labels (e.g., Immediate, Monitor).
FR7	Audio Recording	Record engine sounds for up to 10 seconds using device microphone.
FR8	Sound Classification	Audio analysis classifies sound into predefined fault categories.
FR9	Text Explanation	Faults include plain-language explanations and text-based repair suggestions.
FR10	Tutorial Links	Diagnostic results include links to relevant YouTube tutorial videos.
FR11	History Storage	Diagnostic history saved locally with timestamps and results.
FR12	Maintenance Reminders	Sends reminders for maintenance based on detected faults or schedule.
FR13	Offline Diagnostics	Image and audio diagnostics work offline using preloaded on-device models.
FR14	Online Updates	Updates fault database and tutorials when internet connection is available.
FR15	Vehicle Info Input	Vehicle details (make/model/year) improve diagnostic specificity.
FR16	Multiple Icon Detection	Detects and interprets multiple warning icons in a single captured image.
FR17	Urgency Estimation	Assigns urgency levels (Immediate, Soon, Monitor) to detected issues.
FR18	Report Sharing	Users can save or share diagnostic reports externally.
FR19	Maintenance Tips	Provides preventive tips based on identified problems.
FR20	Mechanic Notes	Mechanics can add professional notes to diagnostic reports (role-restricted).
FR21	User Management	Admins manage users (add, remove, edit profiles) via dashboard.

ID	Functional Requirement	Specification
FR22	Profile Management	Users can edit profile information and set personal preferences.

4.3 Non-Functional Requirements

Performance

- Fast diagnostics: ≤ 5 s (image classification), ≤ 7 s (audio classification)
- Fast UI transitions and authentication: ≤ 3 s
- High throughput: 50 diagnostic sessions/day/user, 100+ concurrent authentication requests
- Resource limits: < 200 MB RAM, $< 30\%$ CPU during analysis, $< 1\%$ battery during standby

Reliability

- Uptime: 99% for online features, 100% for offline core features
- Mean Time Between Failures (MTBF): fewer than 5 crashes per 1000 sessions
- Robust error handling and graceful recovery for: session loss, denied permissions, network failure

Usability

- Simple UI for non-technical users; maximum 3 taps to reach any main function
- Fast onboarding: registration in < 2 minutes, first diagnosis in < 3 minutes
- Accessibility: screen reader support, voice commands, high contrast, WCAG 2.1 AA compliance

Security

- Multi-factor authentication option, secure credential storage, role-based access control (RBAC)
- Audio and image data processed locally on-device; no PII transmitted to external servers
- Explicit runtime permission requests with clear user-facing justification
- Audit trails, integrity checks, and anti-tampering for diagnostic records

Maintainability & Portability

- Modular, well-documented Flutter codebase with layered architecture (UI, business logic, data)
- Full support for Android and iOS, including tablet screen sizes
- Over-the-air (OTA) model updates, versioned database migrations, isolated authentication module

Quality Attributes

- Accuracy: $\geq 90\%$ (visual/dashboard scanning), $\geq 85\%$ (audio classification), $< 5\%$ false positive rate
- Efficiency: ≤ 5 s total diagnosis time, low battery and data consumption
- Privacy: no location tracking, anonymized usage data, compliant with applicable regulations
- Lightweight: optimized for low-bandwidth environments, compressed data transfers

5. Software Requirements Specification (SRS)

Version: 1.0 Status: DRAFT

5.1 Introduction to SRS

5.1.1 Purpose

This Software Requirements Specification (SRS) document describes the functional and non-functional requirements for CheckAR, an AI-based mobile application that enables car owners to diagnose vehicle faults using dashboard warning light recognition and engine sound analysis. It serves as the agreement between stakeholders and the development team and provides a basis for estimating costs, schedules, and validation criteria.

5.1.2 Scope

The application ("CheckAR") will allow users to:

- Register and log in with role-based access (car owners, mechanics, administrators)
- Capture and interpret dashboard warning lights via the device camera
- Record and classify engine sounds via the device microphone
- Receive automated explanations, repair suggestions, maintenance alerts, and video tutorials
- Track maintenance history and receive scheduled reminders
- Operate offline for core diagnostics and online for updated fault data

The application will NOT:

- Replace professional diagnostic tools or mechanic expertise
- Interface directly with vehicle onboard computers (no OBD-II hardware connection)
- Provide real-time monitoring while the vehicle is in motion
- Guarantee 100% accuracy in all diagnostic scenarios

5.1.3 Definitions, Acronyms, and Abbreviations

- App: The CheckAR mobile application
- ML: Machine Learning
- UI/UX: User Interface / User Experience
- MVP: Minimum Viable Product
- OBD-II: On-Board Diagnostics II (vehicle self-diagnostic hardware standard)
- CV: Computer Vision
- PII: Personally Identifiable Information
- API: Application Programming Interface
- 2FA: Two-Factor Authentication
- RBAC: Role-Based Access Control
- MFCC: Mel-Frequency Cepstral Coefficients (audio feature extraction method)

5.1.4 References

- IEEE Std 830-1998: IEEE Recommended Practice for Software Requirements Specifications
- CheckAR_Task2_DataGathering.docx: Survey data gathering and cleaning report
- ISO/IEC 25010:2011: Systems and Software Quality Requirements and Evaluation (SQuaRE)
- ISO 26262: Road Vehicles — Functional Safety Standard
- OWASP Mobile Application Security Verification Standard (MASVS)

5.1.5 Overview

The remainder of this document covers:

- Section 5.2: Overall description of the product, including perspective, functions, user characteristics, constraints, and assumptions
- Section 5.3: Specific requirements, including functional, interface, and non-functional requirements
- Section 5.4: System features with detailed descriptions of core functionality
- Section 5.5: Validation methods, testing strategy, and risk mitigation
- Section 5.6: Appendices with supplementary information

5.2 Overall Description

5.2.1 Product Perspective

CheckAR is a standalone mobile application designed for Android and iOS platforms using the Flutter cross-platform framework. It leverages on-device ML models (TensorFlow Lite) for image and audio classification, with optional cloud connectivity for data updates and enhanced features.

The application relates to existing products as follows:

- Unlike professional OBD-II scanners, CheckAR requires no physical connection to the vehicle
- Compared to existing automotive apps, it uniquely combines visual dashboard scanning and engine audio analysis
- Integrates with third-party content platforms (YouTube Data API) for educational repair materials

5.2.2 Product Functions

- User Authentication: Registration, login, and profile management with role-based access control
- Dashboard Light Scanner: Detect and classify multiple warning lights in a single captured image
- Engine Sound Analyzer: Record, preprocess, and classify engine noises (knocking, squealing, hissing, grinding, clicking)
- Diagnosis Report: Provide cause, urgency level, repair tips, and relevant video tutorial links
- Maintenance Tracker: Log past diagnostics, costs, and schedule automated reminders
- Knowledge Base: Access to common car problems and contextual solutions
- Offline Mode: Core diagnostic features without internet; Online Mode: Fetch latest models, fault databases, and tutorials

5.2.3 User Classes and Characteristics

Primary: Car Owners (Non-Technical)

- Demographics: Ages 18–65, varying technical expertise
- Goals: Quick diagnosis, basic understanding of car issues without specialized knowledge
- Usage frequency: Occasional, primarily when warning lights appear or unusual sounds are detected
- Access level: Standard diagnostic and maintenance features

Secondary: Mechanics / Technicians

- Goals: Quick pre-checks and a client education tool
- Technical expertise: Professional automotive knowledge
- Usage frequency: Regular, across multiple vehicles
- Access level: Enhanced diagnostic features and ability to add professional notes to diagnoses

Tertiary: Content Creators

- Goals: Provide and manage tutorial content linked from diagnostic results
- Access level: Content management interface

System Administrators

- Goals: Manage backend services, update ML models, monitor system performance
- Access level: Full system access, user management, and analytics dashboard

5.2.4 Operating Environment**Mobile Devices:**

- Android 8.0 (API level 26) or higher
- iOS 12.0 or higher
- Minimum 2 GB RAM, 100 MB free storage
- Rear-facing camera with minimum 5 MP resolution
- Device microphone (noise cancellation preferred)

Network:

- Internet connectivity (Wi-Fi or cellular) for online features
- Full offline capability for all core diagnostic features

Backend Services:

- Cloud-based user authentication (Firebase Authentication)
- Cloud-based ML model training infrastructure
- Content delivery network (CDN) for tutorial metadata and model updates
- Secure database for user profiles and preferences

5.2.5 Design and Implementation Constraints**Technical Constraints:**

- On-device ML model size must not exceed 10 MB
- TensorFlow Lite required for on-device ML inference
- Flutter framework for cross-platform development (Android and iOS from a single codebase)
- Diagnostic feedback latency: ≤ 5 seconds
- Battery consumption: $< 5\%$ per full diagnostic session

Business Constraints:

- No personal data stored beyond the scope of the app's core function
- No advertisements in any diagnostic or primary application view
- Initial app download size not to exceed 50 MB

Regulatory Constraints:

- Must include clear disclaimer regarding diagnostic accuracy limitations
- Must comply with applicable data protection regulations (GDPR, CCPA)
- Cannot claim to replace professional mechanical diagnosis

5.2.6 User Documentation

- In-app interactive tutorial for first-time users
- FAQ section covering common use cases and misdiagnosis disclaimers
- Online help center with troubleshooting guides

- Video demonstrations for all key features
- Contextual tooltips on interface elements
- Role-specific guidance for car owners, mechanics, and administrators

5.2.7 Assumptions and Dependencies

- Users grant camera and microphone permissions at runtime
- Access to YouTube Data API for tutorial video retrieval and linking
- Availability of labeled audio and image datasets sufficient for ML model training
- Users possess basic smartphone operation familiarity
- Adequate ambient lighting for dashboard photo capture
- Vehicle engine accessible and audible for sound recording
- Secure authentication service (Firebase) remains available at 99.9% uptime

5.3 Specific Requirements

5.3.1 Functional Requirements

ID	Requirement	Priority	Source
FR1	The app shall provide user registration functionality with email verification.	Must-Have	Security Standards
FR2	The app shall support role-based user authentication (car owner, mechanic, administrator).	Must-Have	Project Design
FR3	The app shall allow users to recover forgotten passwords securely via email.	Must-Have	Security Standards
FR4	The app shall maintain user sessions with appropriate inactivity timeout controls.	Must-Have	Security Standards
FR5	The app shall allow users to capture dashboard images and detect warning light icons.	Must-Have	Survey, Interviews
FR6	The app shall interpret each detected icon and display its meaning and urgency level.	Must-Have	Survey, Expert Interviews
FR7	The app shall record engine sound clips up to 10 seconds using the device microphone.	Must-Have	Survey, Interviews
FR8	The app shall classify engine sounds into predefined fault categories (e.g., knocking, squealing, hissing).	Must-Have	Requirements Analysis
FR9	The app shall provide plain-language explanations and repair suggestions for detected faults.	Must-Have	Survey, Interviews
FR10	The app shall include links to relevant YouTube video tutorials for each diagnosed issue.	Should-Have	Survey, User Feedback
FR11	The app shall store past diagnostics locally with timestamps and results.	Should-Have	Survey, Interviews
FR12	The app shall send maintenance reminders based on user-defined schedules or fault urgency.	Should-Have	Requirements Analysis
FR13	The app shall operate offline for image and audio diagnostics using preloaded on-device models.	Must-Have	Survey, Interviews
FR14	The app shall fetch updated fault databases and tutorial metadata when internet is available.	Should-Have	Survey, Interviews
FR15	The app shall allow users to input vehicle make, model, and year for more specific diagnostics.	Should-Have	Expert Interviews
FR16	The app shall detect multiple warning lights simultaneously in a single captured image.	Must-Have	Project Description
FR17	The app shall estimate maintenance urgency on a three-tier scale (Immediate, Soon, Monitor).	Must-Have	Project Description
FR18	The app shall allow users to save and share diagnostic reports.	Could-Have	User Feedback
FR19	The app shall provide preventive maintenance tips specific to identified issues.	Should-Have	Expert Interviews
FR20	The app shall enable mechanics to add professional notes to diagnoses (mechanic role only).	Should-Have	Project Design

ID	Requirement	Priority	Source
FR21	The app shall provide administrators with full user management capabilities.	Must-Have	Project Design
FR22	The app shall allow users to manage their profile information and notification preferences.	Should-Have	User Feedback

5.3.2 External Interface Requirements

5.3.2.1 User Interfaces

Authentication Screens: Registration form with email verification; login screen with password recovery; role selection during registration; profile management screen.

Home Screen: Quick-access buttons for 'Scan Dashboard' and 'Record Engine Sound'; navigation to history and settings; vehicle profile selector; role-specific feature display.

Dashboard Scanner Screen: Camera viewfinder with framing guide overlay; capture and flash controls; preview and retake options.

Sound Recorder Screen: Recording button with countdown timer; real-time sound level visualization; instructions for optimal recording positioning.

Results Screen: Detected warning icons with labels; audio classification results; explanations, urgency rating, and tutorial links; save/share options; professional notes section (visible only to mechanics).

History Screen: Chronological list of past diagnostics; date/time and issue summary; filtering, search, and comparison functionality.

Admin Dashboard: User management interface; system performance metrics; ML model update controls.

Settings Screen: Notification preferences; data usage controls; vehicle profile management; authentication settings.

5.3.2.2 Hardware Interfaces

- Camera: Rear-facing with autofocus, flash control for low-light, minimum 5 MP
- Microphone: Device microphone, external microphone support, 44.1 kHz sampling at 16-bit depth
- Storage: Local storage for diagnostic history, cache management, secure token storage
- Display: Multiple screen sizes and resolutions, portrait and landscape support, accessible text sizing

5.3.2.3 Software Interfaces

- Authentication: Firebase Authentication, OAuth 2.0 (optional social login), token-based sessions
- ML Inference: TensorFlow Lite with versioned .tflite on-device models
- Tutorial Content: YouTube Data API for video retrieval and embedding
- Local Database: SQLite for diagnostic history; secure storage for credentials; JSON for vehicle specs
- Push Notifications: Firebase Cloud Messaging for maintenance reminders and alerts

5.3.2.4 Communications Interfaces

- HTTPS/TLS for all secure data fetch and API calls
- REST API for backend services and model update delivery
- Encrypted file transfer for model downloads and anonymized feedback uploads (with explicit user consent)

5.3.3 Non-Functional Requirements

Performance Requirements:

- Maximum 5 seconds for image classification; maximum 7 seconds for audio classification
- UI transitions complete within 2 seconds; authentication within 3 seconds
- App launches within 5 seconds; on-device models load within 5 seconds after launch
- Support up to 50 diagnostic sessions per day per user; database queries <100 ms
- Memory usage <200 MB; CPU <30% during analysis; battery <1% during standby

Reliability Requirements:

- 99% uptime for online features; 100% availability for offline core features
- Authentication service availability $\geq 99.9\%$
- Fewer than 5 crashes per 1000 user sessions; authentication failure rate <0.5%
- Graceful degradation for denied permissions, network failures, and session interruptions
- Automatic retry for failed network operations; session recovery after unexpected termination

Usability Requirements:

- Maximum 3 taps to reach any primary function from the home screen
- First-time registration completed in under 2 minutes; first diagnosis under 3 minutes
- Interactive onboarding tutorial for new users
- WCAG 2.1 AA compliance: screen readers, large fonts, high contrast, voice commands

Security Requirements:

- Multi-factor authentication option; password complexity enforcement; account lockout after repeated failures
- Audio and image data processed locally on-device; no PII stored or transmitted remotely
- Anonymized usage analytics only with explicit user opt-in
- Role-based access control (RBAC) for all features; audit trails for administrative actions
- Checksum verification for all model updates to prevent tampering

Maintainability and Portability:

- Modular Flutter codebase with strict separation of UI, business logic, and data layers
- Comprehensive inline code documentation and architecture decision records
- Identical feature sets on Android and iOS, with tablet-optimized layouts
- Over-the-air (OTA) ML model updates with versioned migration strategy

Quality Attributes:

- Accuracy: $\geq 90\%$ for warning light recognition; $\geq 85\%$ for engine sound classification; false positive rate <5%
- Efficiency: $\leq 5s$ total diagnosis; battery consumption <1% per session; data usage <1 MB per online session (excluding video)
- Privacy: no location tracking; all user data anonymized; compliance with GDPR and CCPA
- Lightweight: optimized for sub-2 Mbps environments; compressed model update packages

5.4 System Features

5.4.1 User Authentication System

Description: The user authentication system provides secure registration, login, and role-based access control for car owners, mechanics, and administrators, ensuring appropriate feature access and data security at all times.

Functional Requirements: FR1, FR2, FR3, FR4, FR21, FR22

Technical Implementation:

- Firebase Authentication or equivalent cloud backend service
- Token-based session management with configurable expiration
- Secure credential storage using AES-256 encryption
- Role-Based Access Control (RBAC) enforced at both client and server level
- Email verification workflow with resend capability
- Password policy enforcement (minimum length, complexity, no common passwords)

User Workflow:

- Registration: User selects 'Register', provides email, password, and role; email verification sent; user confirms and completes profile.
- Login: User enters credentials; system authenticates and routes to role-specific home screen.
- Password Recovery: User selects 'Forgot Password'; provides registered email; receives reset link; sets new password.
- Profile Management: User accesses profile from settings; updates personal info, vehicle details, and notification preferences.

5.4.2 Dashboard Warning Light Scanner

Description: The dashboard warning light scanner allows users to capture an image of their vehicle's dashboard and automatically identifies and interprets any active warning lights in the image.

Functional Requirements: FR5, FR6, FR16

Technical Implementation:

- Computer vision model using CNN architecture trained on dashboard warning light datasets
- Image preprocessing pipeline for enhancement across varied lighting conditions
- On-device database of 100+ common warning light symbols across major vehicle manufacturers
- Real-time bounding-box highlighting of detected symbols in the captured image

User Workflow:

- User selects 'Scan Dashboard' from the home screen
- Camera interface appears with framing guide overlay
- User captures dashboard image; app processes and highlights detected lights
- User confirms image or retakes; app displays detailed information per detected light

5.4.3 Engine Sound Analyzer

Description: The engine sound analyzer records engine audio through the device microphone and applies machine learning to classify sounds associated with common mechanical faults.

Functional Requirements: FR7, FR8

Technical Implementation:

- Audio preprocessing pipeline including ambient noise reduction
- Feature extraction using MFCC (Mel-Frequency Cepstral Coefficients)
- Classification model using CNN or LSTM architecture
- Detection of at least 10 common problematic engine sound categories

User Workflow:

- User selects 'Record Engine Sound' from the home screen
- App provides positioning instructions for optimal microphone placement
- User starts recording (5–10 seconds); app processes audio and classifies sounds
- Results display with confidence scores and fault descriptions for each detected category

5.4.4 Diagnosis and Reporting

Description: The diagnosis system combines results from warning light and sound analysis to produce comprehensive fault information, repair suggestions, urgency ratings, and linked educational content.

Functional Requirements: FR9, FR10, FR17, FR19, FR20

Technical Implementation:

- Rule-based fusion system combining visual and audio diagnostic results
- Urgency algorithm based on safety implications and potential component damage
- Curated knowledge base of repair suggestions and preventive maintenance tips
- YouTube Data API integration for contextually relevant tutorial retrieval
- Role-gated notes system allowing mechanics to annotate diagnoses

User Workflow:

- After scan or recording, results page displays all identified issues
- Each issue shows description, probable causes, urgency level, and tutorial link
- User can expand any issue for full detail; mechanics can add professional notes
- User can save report locally or share with a mechanic

5.4.5 Maintenance Tracking

Description: The maintenance tracking system stores diagnostic history and sends reminders for regular maintenance and follow-up actions on previously detected issues.

Functional Requirements: FR11, FR12, FR18

Technical Implementation:

- Local SQLite database for all diagnostic history with cloud sync for registered users
- Firebase Cloud Messaging for scheduled maintenance notification delivery
- Report export functionality (PDF format) and secure sharing via OS-native share sheet

User Workflow:

- User accesses history from the home screen navigation
- Chronological list of past diagnostics is displayed with issue type, urgency, and date
- User can filter by date, issue type, or urgency; set reminders or share with a mechanic

5.4.6 Online / Offline Functionality

Description: CheckAR delivers all core diagnostic functionality without an internet connection, while enabling enhanced features and model/database updates when connectivity is available.

Functional Requirements: FR13, FR14, FR15

Technical Implementation:

- Preloaded TensorFlow Lite models bundled with the app for fully offline operation
- Incremental model updates to minimize cellular data consumption
- Vehicle make/model/year database bundled locally for offline specificity
- Version management system for models and fault databases
- Offline authentication caching mechanism for authenticated users

User Workflow:

- App operates with all core features in offline mode for previously authenticated users
- Background update checks run when connectivity is restored
- User receives notification of available updates with package size; can schedule updates for Wi-Fi

5.5 Validation and Testing

5.5.1 Testing Strategy

- Security Testing: Penetration testing and vulnerability assessment of the authentication system
- Unit Testing: Individual components (authentication, camera pipeline, audio recording, ML classification)
- Integration Testing: End-to-end diagnostic workflows combining visual and audio inputs
- Performance Testing: Response time benchmarks, resource utilization, and battery consumption
- Usability Testing: Representative test groups across car owners and mechanics
- Beta Testing: Limited staged release to gather real-world feedback before full deployment
- Model Validation: Accuracy, precision, recall, and false positive rate evaluation on held-out labeled datasets

5.6 Appendices

A. Glossary

- **Authentication:** The process of verifying a user's identity before granting access
- **RBAC:** Role-Based Access Control — restricting system access based on assigned user roles
- **Diagnosis:** Process of identifying mechanical or electrical faults in a vehicle
- **Model Inference:** Running a trained ML model on new input data to generate predictions
- **Warning Light:** A dashboard indicator lamp signalling the status of a vehicle system
- **Engine Knocking:** An abnormal combustion sound indicative of potential engine issues
- **False Positive:** An incorrect identification of a non-existent fault or problem
- **Model Quantization:** A technique to reduce ML model size for efficient mobile device deployment
- **MFCC:** Mel-Frequency Cepstral Coefficients — audio features used in sound classification
- **Token:** A piece of data representing authentication authorization in a session

B. Analysis Models

- **Use Case Diagram:** Captures major use cases — Register, Login, ScanDashboard, RecordEngineSound, ViewResults, ViewHistory
- **Data Flow Diagram:** Illustrates flow — Authentication → Input Capture → Preprocessing → ML Inference → Report Output
- **Class Diagram:** Relationships among major software components (User, Vehicle, DiagnosisReport, MaintenanceRecord)
- **Sequence Diagram:** Detailed interaction flows during authentication and the full diagnostic process
- **State Diagram:** Application state transitions during a user session

C. Issue Tracking

ID	Issue	Status	Resolution
ISSUE-001	Authentication token expiration handling	In Progress	Implementing refresh token mechanism
ISSUE-002	Model accuracy in low-light dashboard capture	Open	Investigating preprocessing enhancements (histogram equalization, denoising)
ISSUE-003	Audio classification accuracy with background noise	In Progress	Adding ambient noise isolation algorithms to preprocessing pipeline
ISSUE-004	Battery consumption during extended scanning	Resolved	Optimized camera frame rate and CPU scheduling during capture
ISSUE-005	Support for international and regional warning light symbols	Open	Expanding training dataset with international vehicle databases
ISSUE-006	Role transition approval workflow for users upgrading to mechanic role	Open	Designing admin approval workflow and verification process
ISSUE-007	Tutorial relevance accuracy for specific diagnosed faults	In Progress	Refining YouTube API metadata matching algorithm using fault taxonomy

6. Requirements Validation with Stakeholders

6.1 Validation and Risk Mitigation

6.1.1 Stakeholder Engagement

The requirements for CheckAR were gathered and validated through:

- Structured surveys targeting automotive mechanics of varying experience levels (data captured in Task 2: Data Gathering and Cleaning)
- Interviews with car owners representing the primary non-technical user group
- Review sessions with project stakeholders and the course instructor
- Comparative analysis of existing car diagnostic mobile tools

6.1.2 Identified Risks and Mitigation Strategies

Risk	Impact	Likelihood	Mitigation Strategy
Authentication system breach	Critical	Low	Implement industry-standard protocols (OAuth 2.0, 2FA) and schedule regular security audits
Inaccurate fault classification	High	Medium	Iterative ML model retraining using survey feedback and real-world test cases
Limited labeled dataset availability	Medium	High	Use publicly available automotive datasets and crowd-source anonymized diagnostic data with consent
Audio interference in real-world recordings	Medium	Medium	Mandatory noise-reduction preprocessing pipeline in the audio classification module
On-device ML model exceeds size limit	High	Low	Apply model quantization and weight pruning; benchmark regularly against 10 MB constraint
Low adoption due to interface complexity	High	Medium	Prioritize UI/UX simplicity; integrate interactive onboarding and conduct usability testing
Battery drain during extended diagnostic sessions	Medium	Medium	Optimize camera and microphone scheduling; implement power-saving idle modes
False sense of security from app diagnosis	High	Medium	Prominent disclaimers on all result screens; urgency labels directing users to professionals
Privacy concerns regarding vehicle and audio data	Medium	Medium	All processing done on-device; explicit opt-in consent for any anonymized data sharing
User resistance to mandatory registration	Medium	High	Offer a limited guest mode for basic diagnostic features without account creation

General Conclusion

Our structured requirements analysis for CheckAR has delivered a prioritized, unambiguous, and complete set of functional and non-functional requirements grounded in real-world data collected from automotive mechanics and car owners. We have reconciled overlapping feature descriptions, resolved ambiguous scope statements, and verified that no critical stakeholder needs remain undocumented.

Armed with this refined specification, the CheckAR development team can move confidently into system architecture and implementation, knowing that every diagnostic capability, user interaction pattern, and performance target has been rigorously vetted and is positioned to deliver maximum value to car owners and automotive professionals alike.